EECS 281 Practice Midterm: Chapter 17

Hash Tables and Collision Resolution

Modeled on the Fall 2026 Exam 2 Practice (Darden, Paoletti)

Instructions. Select only ONE answer per multiple-choice question. For written questions, follow the line limits and complexity requirements. When asked about memory complexity, consider all possible sources (stack and heap). Questions marked **PAST MIDTERM (EXACT)** are copied word-for-word from the practice exam. Questions marked **NEW (PAST-EXAM PATTERN)** are new and follow the same patterns. Answers start on the page after the questions.

Part I: Past Midterm Questions

1. Collision Resolution — PAST MIDTERM (EXACT)

You are given a hash table with size $M = 10$ and hash function $H(n) = (2n + 3) \bmod M$. Collisions are resolved using quadratic probing. What would the table look like (with an `X` denoting an empty location) after inserting the following elements in the order given? Use $c_1 = 0$ and $c_2 = 1$.

`4, 10, 12, 7, 2`

A) `[X, 4, X, 10, X, X, X, 12, 7, 2]`

B) `[X, 4, 2, 10, X, X, X, 12, 7, X]`

C) `[X, 4, X, 10, X, X, 2, 12, 7, X]`

D) `[X, 4, X, 10, X, X, 2, 12, X, 7]`

2. Hash Table Applications — PAST MIDTERM (EXACT)

In which one of the following situations would you want to use a hash table?

A) You have a classroom full of students each with a 10-digit ID number, and you want to find them by their ID number.

B) You want to find the highest priority thread to execute, each with its own assigned priority.

C) You have a rectangular floor composed of square tiles, and you want to know the color of a tile given a set of coordinates.

D) You are given a set of names, and want to print them out in alphabetical order.

3. Programming: Searching in a Hash Table [12 Points] — PAST MIDTERM (EXACT)

A set is a data structure that can efficiently (less than $O(n)$ time) see if it contains a given element. Given the following definition of a hash table structure:

```
enum class Status { Empty, Occupied, Deleted };
struct HashTable {
  std::vector<int> buckets;
  std::vector<Status> status;
};
```

Suppose this hash table implements the quadratic probing mechanism for collision resolution. When hashing an integer value `val`, the hash function is simply `val % M`, where `M` represents the size of the hash table. Implement the `exists()` function for a set based on a hash table. Use $c_1 = 1$ and $c_2 = 2$.

Requirements: Your solution must implement quadratic probing. Your solution must be $O(n)$ time. You may use $O(1)$ auxiliary space.

Implementation: Use the back of this page as a working area, then rewrite neatly on the front. *Limit:* 16 lines of code (points deducted if longer). You may use any C, C++, or STL function/algorithm that you wish.

```
bool exists(const HashTable &tbl, int val) {
```

Part II: New Questions (Same Patterns)

4. Hash Function Properties

NEW (PAST-EXAM PATTERN)

Which one of the following is **NOT** a valid hash function for `std::string` keys? (Two strings with equal contents are the same key.)

A) `return 0;`

B) `return s.size() * 31;`

C) `return reinterpret_cast<size_t>(&s);` (the memory address of the string object)

D) `return std::accumulate(s.begin(), s.end(), size_t{0});` (the sum of the characters)

5. Compression and Table Size

NEW (PAST-EXAM PATTERN)

A hash table uses division compression, index $= K \bmod M$, with $M = 15$. Every key K that will ever be inserted is a non-negative multiple of 6. How many distinct indices can these keys ever map to?

A) 15

B) 3

C) 5

D) 6

E) 1

6. Linear Probing with Deletion

NEW (PAST-EXAM PATTERN)

A hash table of size $M = 10$ uses hash function $H(n) = (3n + 1) \bmod M$ and resolves collisions with linear probing. Duplicates are not allowed, and erased elements are replaced with a deleted flag (`DEL`). What does the table look like after the following operations, in order?

`insert 3, insert 13, insert 23, insert 6, erase 13, insert 23, insert 33, erase 3, insert 16`

A) `[DEL, DEL, 23, 33, 16, X, X, X, X, 6]`

B) `[16, 23, 23, 33, X, X, X, X, X, 6]`

C) `[16, DEL, 23, 33, X, X, X, X, X, 6]`

D) `[16, 33, 23, X, X, X, X, X, X, 6]`

7. Quadratic Probing

NEW (PAST-EXAM PATTERN)

You are given a hash table with size $M = 11$ and hash function $H(n) = (3n + 2) \bmod M$. Collisions are resolved using quadratic probing, where the i-th probe ($i = 0, 1, 2, \dots$) checks index $(H(n) + c_1 i +$

$c_2 i^2$) mod M. Use $c_1 = 1$ and $c_2 = 1$. What would the table look like after inserting the following elements in the order given?

`5, 16, 27, 3, 14`

A) `[3, 14, X, X, X, X, 5, 16, 27, X, X]`

B) `[3, 27, 14, X, X, X, 5, X, 16, X, X]`

C) `[3, 14, X, X, X, X, 5, 16, X, X, 27]`

D) `[3, X, 14, 27, X, X, 5, X, 16, X, X]`

8. Double Hashing

NEW (PAST-EXAM PATTERN)

A hash table of size $M = 11$ uses double hashing. The j-th probe ($j = 0, 1, 2, \dots$) for key k checks index $(h_1(k) + j \cdot h_2(k)) \bmod M$, where $h_1(k) = k \bmod 11$ and $h_2(k) = 5 - (k \bmod 5)$. What does the table look like after inserting the following keys in the order given?

`14, 25, 36, 3, 58`

A) `[X, X, X, 14, X, 3, X, 36, 25, 58, X]`

B) `[X, X, X, 14, 25, 36, 3, 58, X, X, X]`

C) `[58, X, X, 14, X, 25, X, 36, X, 3, X]`

D) `[X, X, X, 14, X, 3, 58, 36, 25, X, X]`

9. Dynamic Hashing

NEW (PAST-EXAM PATTERN)

An open-addressing hash table starts with $M = 4$, uses $H(k) = k \bmod M$ (where M is the *current* size), and resolves collisions with linear probing. After each insertion, if the load factor $\alpha > 0.5$, the table is doubled and every key is rehashed into the new table, scanning the old table from index 0 to $M - 1$. What does the table look like after inserting `9, 17, 6, 14` in that order?

A) `[X, 9, 17, 6, X, X, 14, X]`

B) `[X, 9, 17, X, X, X, 6, 14]`

C) `[14, 9, 17, 6]`

D) `[X, 9, 17, X, X, X, 14, 6]`

10. Collision Resolution Facts

NEW (PAST-EXAM PATTERN)

Which of the following statements about hash tables is TRUE?

A) Slots holding a deleted flag count toward N when computing the load factor $\alpha = N/M$.

B) With quadratic probing, an insert always finds an open slot as long as at least one slot in the table is empty.

C) When a hash table is resized, each element can be copied to the same index in the new, larger table.

D) Double hashing reduces secondary clustering because keys that hash to the same index can follow different probe sequences.

E) In a hash table using separate chaining, the load factor α can never exceed 1.

11. Hash Table Applications

NEW (PAST-EXAM PATTERN)

In which one of the following situations would a hash table be the best choice?

A) You want to list every employee whose salary is between \$50,000 and \$80,000.

B) A website receives 10 million sign-ups and must check, for each one, whether that username is already taken.

C) You repeatedly need to remove the job with the earliest deadline from a set of jobs.

D) You need to look up the name of the weekday for an integer from 0 to 6.

E) You want to print every distinct word in a book in alphabetical order.

12. STL Unordered Map

NEW (PAST-EXAM PATTERN)

What does the following code print?

```
std::unordered_map<std::string, int> m;
m["eecs"] = 281;
m.insert({"eecs", 370});
m.insert({"math", 217});
if (m.find("stats") == m.end())
  m.emplace("stats", 250);
if (m["phys"] > 0)
  m["eecs"] = 482;
m.erase("math");
std::cout << m.size() << " " << m["eecs"] << " " << m.count("phys");
```

A) `2 281 0`

B) `3 281 1`

C) `3 370 1`

D) `4 281 1`

E) `2 482 0`

13. Programming: Inserting into a Hash Table [12 Points]

NEW (PAST-EXAM PATTERN)

Use the same `Status` and `HashTable` definitions as Question 3. The table now resolves collisions with **double hashing**: the j-th probe ($j = 0, 1, 2, \dots$) for `val` checks index $(h_1 + j \cdot h_2) \bmod M$, where $h_1 =$ `val % M` and $h_2 = 7 -$ `(val % 7)`. Assume all values are non-negative and M is a prime greater than 7.

Implement `insert()`. The table does not allow duplicates: if `val` is already in the table, return `false` and change nothing. Otherwise, store `val` in the *first* `Deleted` or `Empty` slot along its probe sequence and return `true`. If there is no such slot, return `false`.

Requirements: $O(n)$ time, $O(1)$ auxiliary space. *Limit:* 18 lines of code.

```
bool insert(HashTable &tbl, int val) {
```

14. Programming: Counting Pair Sums [8 Points]

NEW (PAST-EXAM PATTERN)

Given a vector of integers `nums` and an integer `target`, return the number of index pairs (i, j) with $i < j$ and `nums[i] + nums[j] == target`. For example, `nums = [1, 5, 3, 3, 3]` and `target = 6` returns `4`: the pair $(1, 5)$ plus three pairs of 3s. The input is not sorted.

Requirements: average-case $O(n)$ time; you may use $O(n)$ auxiliary space. *Limit:* 12 lines of code. You may use any C, C++, or STL function/algorithm that you wish.

```
size_t count_pairs(const std::vector<int> &nums, int target) {
```

Answer Key

1. C `[X, 4, X, 10, X, X, 2, 12, 7, X]`. H: $4 \to 1$, $10 \to 3$, $12 \to 7$, $7 \to 7$. 7 collides at 7, so it probes $7+1^2 = 8$. Then 2 hashes to 7: it probes 8 (taken), $7+4 = 11 \to 1$ (taken), and $7+9 = 16 \to 6$ (empty).

2. A. Looking up students by a sparse 10-digit key is the textbook hash table use. B is a priority queue, C is direct addressing (a 2-D array), and D needs sorted order (sort it or use a BST).

3. Probe i checks `(val % M + 1*i + 2*i*i) % M`. Stop with `false` at an `Empty` slot. Skip `Deleted` slots. Return `true` on an `Occupied` slot that holds `val`. Cap the loop at M probes so it is $O(n)$.

```
bool exists(const HashTable &tbl, int val) {
  size_t M = tbl.buckets.size();
  size_t h = static_cast<size_t>(val) % M;   // assumes val >= 0
  for (size_t i = 0; i < M; ++i) {
    size_t idx = (h + i + 2 * i * i) % M;
    if (tbl.status[idx] == Status::Empty) return false;
    if (tbl.status[idx] == Status::Occupied && tbl.buckets[idx] == val)
      return true;
  }
  return false;
}
```

Common mistakes: stopping at a `Deleted` slot, or looping forever when the table has no `Empty` slot.

4. C. A hash function must be deterministic: equal keys must always give the same hash. Two different `std::string` objects holding the same text have different addresses, so you could never find the key again. A (always 0) is valid but terrible, B and D are valid (equal strings give equal results), though they collide often.

5. C (5). $K = 6m$ shares the factor $\gcd(6, 15) = 3$ with M, so every index is a multiple of 3: $\{0, 3, 6, 9, 12\}$. This is why M should be prime: with $M = 17$, all 17 indices could be used.

6. D `[16, 33, 23, X, X, X, X, X, X, 6]`. H: $3, 13, 23, 33 \to 0$ and $6, 16 \to 9$. The first four inserts give `[3, 13, 23, X, ..., 6]`. Erase 13 marks slot 1 `DEL`. Insert 23 probes 0, 1 (`DEL`, keep going), 2 (hit), so there is no insert. Insert 33 probes 0, 1, 2, then stops at empty slot 3. It is not present, so it goes into the first `DEL`/empty slot seen, which is 1. Erase 3 marks slot 0 `DEL`. Insert 16 probes 9, wraps to 0 (`DEL`), 1, 2, then stops at empty slot 3 and goes into slot 0. (B is what you get by emptying slots instead of using `DEL`. A comes from inserting at the first *empty* slot and never reusing `DEL`. C reuses a `DEL` slot for 16 but not for 33.)

7. B `[3, 27, 14, X, X, X, 5, X, 16, X, X]`. With $c_1 = c_2 = 1$ the probe offsets are $0, 2, 6, 12, \ldots$. H: $5, 16, 27 \to 6$ and $3, 14 \to 0$. 5 goes to 6. 16 goes to $6+2 = 8$. 27 tries 6 and 8, then $6+6 = 12 \to 1$. 3 goes to 0. 14 goes to $0 + 2 = 2$. (A is linear probing, and C uses $c_1 = 0$.)

8. A `[X, X, X, 14, X, 3, X, 36, 25, 58, X]`. Every key has $h_1 = 3$. 14 goes to 3. 25: $h_2 = 5$, so it lands at 8. 36: $h_2 = 4$, so it lands at 7. 3: $h_2 = 2$, so it lands at 5. 58: $h_2 = 2$, so it tries 5 and 7 (both taken), then lands at 9. (B is linear probing. C uses h_1 instead of the key to compute h_2.)

9. B `[X, 9, 17, X, X, X, 6, 14]`. With $M = 4$: 9 goes to 1, and 17 goes to 2 ($\alpha = 0.5$, so no resize). 6 goes to 3, so $\alpha = 0.75 > 0.5$ and the table resizes to $M = 8$. Rehashing gives 9 at 1, 17 at

1 then 2, and 6 at 6. Then 14 maps to 6 (taken), so it goes to 7, and $\alpha = 0.5$ means no resize. (A copies slots without rehashing, and C never resizes.)

10. D. A is false: deleted flags do not count toward N. B is false: quadratic probing only reaches about half the slots and can cycle forever when $\alpha > \frac{1}{2}$. C is false: indices depend on M, so every key must be rehashed. E is false: with chaining, α is the average list length and can exceed 1.

11. B. A membership check ("have I seen this?") on a huge key space runs in average $\Theta(1)$ with `unordered_set`. A is a range query (a sorted structure or BST). C is a priority queue. D is direct addressing (a plain array). E needs sorted order.

12. B `3 281 1.` `insert` never overwrites, so `"eecs"` stays 281. `"stats"` is added by `emplace`. `m["phys"]` *inserts* `{"phys", 0}`, and $0 > 0$ is false, so 482 is never assigned. After erasing `"math"` the keys are eecs, stats, and phys, so the size is 3. `count("phys")` is 1.

13. Probe like a search. Return `false` on a hit. Remember the first non-`Occupied` slot, and stop at the first `Empty` slot (the key cannot be past it). Because M is prime and $1 \le h_2 \le 7 < M$, the M probes visit every slot exactly once.

```
bool insert(HashTable &tbl, int val) {
  size_t M = tbl.buckets.size();
  size_t h1 = val % M, h2 = 7 - (val % 7);
  size_t spot = M;                                  // M = no free slot seen yet
  for (size_t j = 0; j < M; ++j) {
    size_t idx = (h1 + j * h2) % M;
    if (tbl.status[idx] == Status::Occupied) {
      if (tbl.buckets[idx] == val) return false;    // duplicate
    } else {
      if (spot == M) spot = idx;                    // first Deleted/Empty
      if (tbl.status[idx] == Status::Empty) break; // val can't be later
    }
  }
  if (spot == M) return false;                      // table full
  tbl.buckets[spot] = val;
  tbl.status[spot] = Status::Occupied;
  return true;
}
```

Common mistakes: inserting into the first `Deleted` slot without first checking the rest of the sequence for a duplicate, or stopping the search at a `Deleted` slot.

14. Store a count of each value seen so far. For each element `x`, every earlier copy of `target - x` forms one pair. This is two-sum (Example 17.7) with counting. It runs in average $\Theta(n)$ time and $O(n)$ space. Using `find` keeps the lookup from inserting unneeded keys.

```
size_t count_pairs(const std::vector<int> &nums, int target) {
  std::unordered_map<int, size_t> seen;   // value -> times seen so far
  size_t pairs = 0;
  for (int x : nums) {
    auto it = seen.find(target - x);
    if (it != seen.end()) pairs += it->second;
    ++seen[x];
  }
  return pairs;
}
```

Trace on `[1, 5, 3, 3, 3]` with target 6: $0 \to 1$ (pair 1, 5) $\to 1 \to 2 \to 4$. Incrementing `seen[x]` *after* the lookup is what keeps $i < j$, so an element never pairs with itself. A nested loop would be $\Theta(n^2)$ and lose points.